FIELD NOTES

Shopify Field Guide

Shopify order, inventory and payout reconciliation fixes.

36 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 36 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/shopify-fixes
Guides: allanninal.dev/shopify
All 14 repos: github.com/allanninal

The fixes

1 **3PL fulfillment out of sync**

The warehouse packed the box, handed it to the carrier, and even has a tracking number to prove it. But Shopify still shows the fulfillment order as In progress, so the order looks unfulfilled in reports, customers do not get a shipping email, and staff waste time trying to fulfill something that already shipped. Here is why the two sides drift apart and a small script that finds the orders stuck this way and flags them for review, without touching a state it is not allowed to change.

`flag_fulfillment_out_of_sync.py`

https://www.allanninal.dev/shopify/3pl-fulfillment-out-of-sync/

https://github.com/allanninal/shopify-fixes/tree/main/3pl-fulfillment-out-of-sync

---

2 **Shopify authorized payments that expire before capture**

Your store captures payments manually, maybe so you can confirm stock or screen an order before you take the money. At checkout Shopify only authorizes the card, so the order sits at an Authorized status with the money held. Then it gets forgotten. About seven days later the hold expires, the money is released, and you have an order you were never paid for. Here is why it happens and a small job that captures the eligible orders before the window closes.

`capture_authorized.py`

https://www.allanninal.dev/shopify/authorized-payment-expires-uncaptured/

https://github.com/allanninal/shopify-fixes/tree/main/authorized-payment-expires-uncaptured

---

3 **Shopify Available drifted from the real on-hand count**

A cycle count says forty units are on the shelf. Shopify says Available is thirty one. Nobody changed anything on purpose, a POS miscount here, a damaged unit written off there, a warehouse webhook that never arrived, and now the number on the store and the number on the shelf quietly disagree. Here is why that gap opens up and a small script that closes it with a compare-and-set write, so it never overwrites a change that just happened.

`reconcile_available.py`

https://www.allanninal.dev/shopify/available-drifted-from-real-on-hand/

https://github.com/allanninal/shopify-fixes/tree/main/available-drifted-from-real-on-hand

---

4 **Billing runs on a cancelled contract**

A customer cancels their subscription, and a day or two later a fresh charge shows up anyway. Support says the contract is cancelled, the customer says they were billed, and both are right. A scheduled billing attempt that was already in flight went ahead and created an order after the contract's status had already moved to cancelled. Here is why Shopify lets that attempt slip through and a small script that finds the orders it created so you can catch them and make it right.

`flag_attempts_after_cancel.py`

https://www.allanninal.dev/shopify/billing-runs-on-a-cancelled-contract/

https://github.com/allanninal/shopify-fixes/tree/main/billing-runs-on-a-cancelled-contract

---

5 **Bulk stock true-up after a bad import**

Someone uploaded a stock file, and now half the catalog shows the wrong number on hand. Maybe the wrong column got mapped to quantity, maybe an old export got uploaded by mistake, maybe a sync app ran twice. Whatever happened, Shopify does not know the file was bad, it just wrote down whatever the file said. Here is why a bad import spreads wrong counts across hundreds of items at once, and a small script that compares the live store against a trusted snapshot and re-sets only what is actually wrong, in safe batches.

`true_up_inventory.py`

https://www.allanninal.dev/shopify/bulk-stock-true-up-after-a-bad-import/

https://github.com/allanninal/shopify-fixes/tree/main/bulk-stock-true-up-after-a-bad-import

---

6 **Chargeback pulled funds, order still Paid**

A customer disputes a charge with their bank. The card network pulls the money out of your next payout almost immediately. But open Shopify admin and the order still says Paid, sitting there like nothing happened. Support answers questions about it, finance reports it as revenue, and nobody notices the funds are gone until the payout report comes up short. Here is why Shopify leaves the order looking fine and a small script that writes the dispute state onto the order so it does not.

`flag_open_chargebacks.py`

https://www.allanninal.dev/shopify/chargeback-pulled-funds-order-still-paid/

https://github.com/allanninal/shopify-fixes/tree/main/chargeback-pulled-funds-order-still-paid

---

7 **Shopify subscription contract has no valid payment method**

A customer removed their card, a bank revoked the stored token, or a contract was created and nobody ever attached a way to pay. Shopify will still try to bill the next cycle, and it will still fail, but nothing shouts about it until a customer notices their subscription quietly stopped shipping. Here is why a contract ends up with no valid payment method and a small script that finds the ones that cannot bill and flags them so you can ask the buyer for a new card.

`find_contracts_missing_payment_method.py`

https://www.allanninal.dev/shopify/contract-has-no-valid-payment-method/

https://github.com/allanninal/shopify-fixes/tree/main/contract-has-no-valid-payment-method

---

8 **Dunning never retries a failed charge**

A customer's card declined, or the bank flagged the charge, or the gateway timed out for a second. Shopify marks the subscription contract's last payment as failed and moves on. Nothing tries the charge again. The subscription just sits there, unpaid, until a human happens to notice. Here is why that gap exists and a small script that retries failed charges on a safe backoff, so recoverable revenue actually gets recovered.

`retry_failed_billing.py`

https://www.allanninal.dev/shopify/dunning-never-retries-a-failed-charge/

https://github.com/allanninal/shopify-fixes/tree/main/dunning-never-retries-a-failed-charge

---

9 **Shopify double charges from duplicate transactions**

One order, two charges. The customer paid once but their card shows the amount twice, and now you have an angry email and a chargeback risk. It usually comes from a retry after a timeout, a double click on a manual capture, or an app that captured the same order more than once. Here is why it happens and a small job that finds the duplicate charges and refunds the extra in a safe way.

`find_duplicate_charges.py`

https://www.allanninal.dev/shopify/duplicate-charge-transactions/

https://github.com/allanninal/shopify-fixes/tree/main/duplicate-charge-transactions

---

10 **Duplicate customers for one email in Shopify**

The same shopper checked out as a guest, then made an account, then typed their email with a different capital letter one busy Friday. Now Shopify shows three customer records for one person, each with its own slice of order history, store credit, and marketing consent. Here is why Shopify splits them apart and a small script that finds the duplicates and merges them back into one record, safely.

`find_duplicate_customers.py`

https://www.allanninal.dev/shopify/duplicate-customers-for-one-email-shopify/

https://github.com/allanninal/shopify-fixes/tree/main/duplicate-customers-for-one-email-shopify

---

11 **Shopify duplicate renewal charges from a retry with no idempotency key**

A renewal times out, or a webhook fires twice, so the caller retries the billing attempt. It generated a new idempotency key instead of reusing the old one, so Shopify treated the retry as a brand new sale. Now the subscriber has two orders and two successful charges for one billing cycle. Here is why a retry turns into a real duplicate and a small script that finds the extra charge on each contract and flags it for a refund review.

`find_duplicate_renewals.py`

https://www.allanninal.dev/shopify/duplicate-renewal-charges/

https://github.com/allanninal/shopify-fixes/tree/main/duplicate-renewal-charges

---

12 **Duplicate webhook deliveries run twice**

You set up a webhook, it fires when an order is paid, and everything works, until one day it fires twice for the same order. Store credit gets granted twice. A confirmation email goes out twice. A fulfillment gets created twice. Nothing in your app crashed, Shopify just did exactly what it is supposed to do: retry a delivery it was not sure you received. Here is why the same event arrives more than once and a small script that finds the orders a duplicate delivery hit so you can clean them up and stop it happening again.

`dedupe_webhook_deliveries.py`

https://www.allanninal.dev/shopify/duplicate-webhook-deliveries-run-twice/

https://github.com/allanninal/shopify-fixes/tree/main/duplicate-webhook-deliveries-run-twice

---

13 **External id metafield dropped on create**

Your linking key vanished on order create. A warehouse system, a marketplace, or an ERP relies on a metafield to match a Shopify order back to its own record, and on some orders that metafield never gets written. The two systems quietly lose track of each other, and nobody notices until a fulfillment or a refund cannot be matched. Here is why it happens and a small script that audits recent orders and sets the metafield back only where it is truly missing or wrong.

`repair_external_id_metafield.py`

https://www.allanninal.dev/shopify/external-id-metafield-dropped-on-create/

https://github.com/allanninal/shopify-fixes/tree/main/external-id-metafield-dropped-on-create

---

14 **Fulfillment stuck On hold**

A hold was placed on a fulfillment order for a real reason: a fraud flag, a bad address, an item that was out of stock. Someone fixed the reason. Nobody ever released the hold. The fulfillment order still shows On hold, the order still will not ship, and it will sit there forever unless something calls the release. Here is why Shopify leaves it stuck and a small script that releases the specific holds that are actually safe to clear, and nothing else.

`release_fulfillment_holds.py`

https://www.allanninal.dev/shopify/fulfillment-stuck-on-hold/

https://github.com/allanninal/shopify-fixes/tree/main/fulfillment-stuck-on-hold

---

15 **Shopify high-risk orders that ship without review**

Shopify quietly scores every order for fraud risk. But scoring is not stopping. A high-risk order sits in the same queue as everything else, and if nobody notices the little risk flag, it gets picked, packed, and shipped. Weeks later it comes back as a chargeback and you lose the goods and the money. Here is why these orders slip through and a small job that tags them for review before they ship.

`flag_high_risk.py`

https://www.allanninal.dev/shopify/high-risk-orders-unactioned/

https://github.com/allanninal/shopify-fixes/tree/main/high-risk-orders-unactioned

---

16 **Missed webhooks with no backfill**

Your app went down, or a deploy misconfigured the endpoint, or a certificate expired quietly over a long weekend. Shopify kept trying to deliver its webhooks the whole time, but its patience runs out. Once your outage crosses that limit, the events that fired during it are gone for good, and no setting brings them back. Here is why the gap is unrecoverable and a small script that polls what changed and applies it anyway.

`backfill_missed_webhooks.py`

https://www.allanninal.dev/shopify/missed-webhooks-with-no-backfill/

https://github.com/allanninal/shopify-fixes/tree/main/missed-webhooks-with-no-backfill

---

17 **Shopify negative inventory from overselling**

A popular item is set to keep selling when it runs out, a rush of orders comes in, and now its stock reads minus fourteen. Nothing on a shelf can be negative, but Shopify shows it because you sold past zero. That negative number then poisons your reports, your reorder points, and any tool that reads stock. Here is why it happens and a small job that finds the oversold variants and resets them to zero in a safe way.

`fix_negative_inventory.py`

https://www.allanninal.dev/shopify/negative-inventory-oversell/

https://github.com/allanninal/shopify-fixes/tree/main/negative-inventory-oversell

---

18 **Next billing date drifts**

A subscription was paused for a week, or someone typed a new date into the admin, or one billing attempt never ran. The contract's billing policy still says charge every 30 days, but the nextBillingDate field on the contract no longer agrees with that. The stored date and the real cycle disagree, and until you catch it, invoices go out on the wrong day or not at all. Here is why the drift happens and a small script that compares the two and realigns the schedule.

`fix_next_billing_date.py`

https://www.allanninal.dev/shopify/next-billing-date-drifts/

https://github.com/allanninal/shopify-fixes/tree/main/next-billing-date-drifts

---

19 **On-hand vs available vs committed**

Someone read the physical stock count, called it the sellable count, and shipped a number to the storefront. The two are not the same thing. Shopify keeps on_hand, committed, and available as three separate quantities on purpose, and the moment code or a manual edit treats the raw stock as if it were free to sell, the store either oversells what is already promised to someone else, or hides stock that a customer could have bought. Here is what the real quantity names mean and a small script that reports the difference wherever it shows up.

`find_available_vs_committed_drift.py`

https://www.allanninal.dev/shopify/on-hand-vs-available-vs-committed/

https://github.com/allanninal/shopify-fixes/tree/main/on-hand-vs-available-vs-committed

---

20 **Shopify orders stuck Unfulfilled past your shipping SLA**

A customer paid three days ago and their order still has not shipped. It is not on hold, nothing is wrong with it, it just slipped through. Shopify shows it as Unfulfilled like a hundred others and never raises its hand. The first person to notice is usually the customer, asking where their order is. Here is why paid orders quietly sit unfulfilled and a small job that flags the overdue ones for review before they become complaints.

`flag_stale_unfulfilled.py`

https://www.allanninal.dev/shopify/orders-stuck-unfulfilled/

https://github.com/allanninal/shopify-fixes/tree/main/orders-stuck-unfulfilled

---

21 **Shopify orders stuck Unpaid after a payment made outside checkout**

The money came in by bank transfer, or the driver collected cash on delivery, or a wholesale customer paid an invoice. The order is real and paid. But Shopify still shows it as Pending, so it does not count as revenue, and some fulfillment and shipping rules will not let it move. Here is why Shopify leaves these orders behind and a small script that finds the confirmed ones and marks them paid in a safe way.

`mark_paid.py`

https://www.allanninal.dev/shopify/orders-stuck-unpaid-mark-as-paid/

https://github.com/allanninal/shopify-fixes/tree/main/orders-stuck-unpaid-mark-as-paid

---

22 **Overselling from concurrent writes**

Two inventory updates fired within the same second, for the same variant, at the same location. Both read the available count as five. Both decided it was fine to write a new number. One write landed, then the other landed right on top of it and erased the first one. The count that Shopify shows now has no memory of that first sale, so the store sells more than it has. Here is why the race happens and a small script that closes it with compareQuantity.

`reconcile_inventory.py`

https://www.allanninal.dev/shopify/overselling-from-concurrent-writes/

https://github.com/allanninal/shopify-fixes/tree/main/overselling-from-concurrent-writes

---

23 **Paid checkouts stranded as abandoned**

A customer finishes checkout, the payment is captured, and then nothing. No order shows up. Shopify quietly files the whole thing under abandoned checkouts, right next to the carts people genuinely walked away from, so a real, paid customer gets treated like a browser who left. Here is why the order write can fail after the money moves, and a small script that finds the stranded checkouts and hands them to a human to reconcile.

`find_stranded_checkouts.py`

https://www.allanninal.dev/shopify/paid-checkouts-stranded-as-abandoned/

https://github.com/allanninal/shopify-fixes/tree/main/paid-checkouts-stranded-as-abandoned

---

24 **Partial refund leaves the tax untouched**

A customer returned one item from a bigger order. Someone typed the item's price into the refund box, hit refund, and moved on. The item's price came back, but the tax the customer paid on that item never did. Shopify does not refund tax on its own, it only refunds what you tell it to. Here is why that gap opens up and a small script that finds it and closes it.

`find_untaxed_partial_refunds.py`

https://www.allanninal.dev/shopify/partial-refund-leaves-the-tax-untouched/

https://github.com/allanninal/shopify-fixes/tree/main/partial-refund-leaves-the-tax-untouched

---

25 **Presentment vs settlement currency**

The buyer paid in pounds, or euros, or pesos. Your payout lands in dollars. Both numbers are correct, they just live in different currencies, and the order carries both. The trouble starts when a report, a spreadsheet, or a support reply reads only one side of that pair and treats it as the whole story. Here is why the two amounts differ, and a small script that makes the exchange explicit and flags the orders where it does not add up.

`find_currency_mismatch.py`

https://www.allanninal.dev/shopify/presentment-vs-settlement-currency-shopify/

https://github.com/allanninal/shopify-fixes/tree/main/presentment-vs-settlement-currency-shopify

---

26 **Processing fee not recorded on the order**

A customer pays fifty dollars, and the order says fifty dollars. But Shopify Payments keeps a slice of that as a processing fee before the payout ever reaches your bank, and the order never mentions it. Every revenue report built from order totals is quietly counting money you never kept. Here is why the fee goes missing and a small script that pulls it from the transaction and writes it back onto the order.

`record_processing_fee.py`

https://www.allanninal.dev/shopify/processing-fee-not-recorded-on-the-order/

https://github.com/allanninal/shopify-fixes/tree/main/processing-fee-not-recorded-on-the-order

---

27 **Reconcile Shopify orders against payouts**

The bank deposit lands, and it never equals the order totals for the day. Fees came out, a refund got subtracted, a few orders from yesterday rode along, and the currencies do not all match. Finance ends up trying to match a single deposit number to a page of orders by eye, and gives up halfway through. Here is why a payout never equals order totals and a small script that ties every payout to its own balance transactions to the cent.

`reconcile_payout_orders.py`

https://www.allanninal.dev/shopify/reconcile-orders-against-payouts/

https://github.com/allanninal/shopify-fixes/tree/main/reconcile-orders-against-payouts

---

28 **Refund exists but the money never moved**

Support marked the order refunded. The customer is staring at their bank statement asking where the money is. Shopify shows a refund object right there on the order, so it looks done. But the refund object is only a record of an attempt, and the gateway transaction underneath it can fail, error out, or sit on Pending forever. Here is why a refund can exist and go nowhere, and a small script that finds the stuck ones so you can chase them down before the customer does it for you in a chargeback.

`find_stuck_refunds.py`

https://www.allanninal.dev/shopify/refund-exists-but-the-money-never-moved/

https://github.com/allanninal/shopify-fixes/tree/main/refund-exists-but-the-money-never-moved

---

29 **Shopify subscription renewals that fail on an expired card**

A subscriber has been happy for a year. Their card quietly expires. On the next renewal the charge fails, the subscription goes past due, and unless your dunning saves it, they churn without ever meaning to leave. The frustrating part is that this one is predictable. You can see the expiry coming. Here is why renewals fail on an old card and a small job that emails those customers to update it before the renewal date.

`notify_expired_cards.py`

https://www.allanninal.dev/shopify/renewal-fails-expired-card/

https://github.com/allanninal/shopify-fixes/tree/main/renewal-fails-expired-card

---

30 **Routed to an out-of-stock location**

The order looks fine everywhere except the one place it matters: the location it got assigned to has nothing to pick. Staff open it in the warehouse and there is no stock, so it just sits there, OPEN or IN_PROGRESS, going nowhere. Here is why Shopify routes orders this way and a small script that finds the ones stuck on a dead location and moves them to one that can actually ship.

`reroute_out_of_stock_fulfillment.py`

https://www.allanninal.dev/shopify/routed-to-an-out-of-stock-location/

https://github.com/allanninal/shopify-fixes/tree/main/routed-to-an-out-of-stock-location

---

31 **Shopify selling plan deleted after an app uninstall**

A subscriptions app was doing its job, then someone uninstalled it to switch providers, or because a trial ended, or during a store cleanup. Within minutes, the subscribe and save button was gone from a bunch of products. Nothing else changed. The product is still there, the price is still there, but the option to buy it as a subscription vanished. Here is why removing the app took the selling plan group with it, and a small script that finds the affected products and rebuilds a group you own.

`rebuild_selling_plan.py`

https://www.allanninal.dev/shopify/selling-plan-deleted-after-app-uninstall/

https://github.com/allanninal/shopify-fixes/tree/main/selling-plan-deleted-after-app-uninstall

---

32 **Test and Bogus Gateway orders in live data**

Someone tested checkout on the live theme. A developer ran through the flow with the Bogus Gateway during a project. A staff member placed a practice order to see what the customer gets. None of that money is real, but every one of those orders sits in the same Orders list as your paying customers, in the same export, and in the same revenue number, unless something separates them out.

`flag_test_orders.py`

https://www.allanninal.dev/shopify/test-and-bogus-gateway-orders-in-live-data/

https://github.com/allanninal/shopify-fixes/tree/main/test-and-bogus-gateway-orders-in-live-data

---

33 **Shopify orders whose transactions do not match the total**

Every order has a running tally of money: what was captured, what was refunded, what is left. Those pieces should add up to the amount you actually received. When they do not, the order's ledger is quietly wrong, and it drags your revenue and refund reports off with it. The gap is small per order and easy to miss, which is exactly why it grows. Here is why the numbers drift and a small job that finds the orders that do not tie out and flags them for review.

`find_ledger_mismatch.py`

https://www.allanninal.dev/shopify/transactions-total-mismatch/

https://github.com/allanninal/shopify-fixes/tree/main/transactions-total-mismatch

---

34 **Shopify untracked items sell without limit**

A product keeps selling no matter how many orders come in. There is no oversold warning, no negative number, nothing. That is because the variant was never tracking inventory in the first place, so Shopify has no stock count to run out of. The buy button just stays on forever. Here is why that happens and a small script that finds the variants quietly running without a stock count and turns tracking on for the ones that should have one.

`fix_untracked_items.py`

https://www.allanninal.dev/shopify/untracked-items-sell-without-limit/

https://github.com/allanninal/shopify-fixes/tree/main/untracked-items-sell-without-limit

---

35 **Webhook HMAC check keeps failing**

Shopify is sending the webhook. Your endpoint is receiving it. And yet every single request fails your HMAC check, so you reject events you should be accepting. Almost always the cause is simple and invisible: something read the raw body, turned it into a JSON object, and your verification code hashed that object instead of the bytes Shopify actually signed. Here is why that happens and a small, tested check that verifies against the raw bytes every time.

`verify_webhook_hmac.py`

https://www.allanninal.dev/shopify/webhook-hmac-check-keeps-failing/

https://github.com/allanninal/shopify-fixes/tree/main/webhook-hmac-check-keeps-failing

---

36 **Shopify webhook subscription auto-deleted after too many failed deliveries**

Your endpoint went down for a while, or it started returning errors, and Shopify quietly gave up on it. The webhook subscription is gone, no email arrived to say so, and the app keeps running while it silently stops hearing about orders, fulfillments, or refunds. Here is why Shopify removes a subscription on its own and a small script that finds the gap and recreates it safely.

`recreate_missing_webhooks.py`

https://www.allanninal.dev/shopify/webhook-subscription-auto-deleted/

https://github.com/allanninal/shopify-fixes/tree/main/webhook-subscription-auto-deleted

---

MIT licensed. Free to use, change and ship. Written by Allan Niñal — allanninal.dev